Hi everyone, and thank you for being here. My name is Shubham Singh, and the topic of my talk is **'Behind the Merge',** I will talk about my LFX mentorship work and my journey with open source.

So, I'll start with a brief introduction to the project I contributed to which is **KubeEdge**, and then we will talk about the issue I worked on during my LFX Mentorship.

After that, I'll walk you through my journey as a student diving into open source. I'll cover how it helped me in learning about new technologies, how it helped me get internships, and get other great opportunities like the linux foundation scholarship, or presenting a session at KubeCon and a lot of other opportunities.

Finally, I'll talk about the mistakes I made along the way and share some lessons I learned from that journey. So, let's get started.

Introduction

A little bit about myself, My name is Shubham, and I am working as an SRE intern at Obmondo. I am a computer science student. I am a CKA and I did my LFX with KubeEdge last year and I am also an org member at KubeEdge.

KubeEdge Intro

So the project I did my LFX mentorship with is KubeEdge. KubeEdge is Kubernetes but for edge. KubeEdge is an open-source platform designed to extend Kubernetes from the cloud to the edge, aim of KubeEdge is to enable Edge Computing and to enable seamless collaboration between cloud and edge. The focus is consistent experience of applications, resources, data etc. between cloud and edge.

In data centers, we have mature infrastructure and services. But at the edge? Things are still developing. That's why KubeEdge was open-sourced and donated to CNCF—to bring community-driven innovation and open source development.

KubeEdge Architecture

So this is the broad level architecture of KubeEdge and how it enables seamless cloud-edge coordination. Starting at the cloud level, we have the CloudCore, which acts as the control plane, similar to Kubernetes. It manages both cloud nodes and edge nodes within the same cluster. The communication between CloudCore and EdgeCore happens through a duplex channel, using WebSocket by default, though QUIC is also supported for better performance in low-latency environments. This allows the system to remain operational even over unstable networks. On the edge, we run EdgeCore, which includes lightweight optimizations, reducing resource usage to as little as 70-80 MB of memory. This ensures nodes can function independently, even offline, with local data persistence. EdgeCore initiates the communication with CloudCore, setting up a two-way channel. This solves challenges like working behind firewalls or gateways where public IPs may not be available. The edge nodes run containers managed by Kubernetes, leveraging container runtimes like Docker and containerd. We also use EdgeMesh to handle networking between edge pods. For devices, KubeEdge introduces a Device Mapper (DMI). Devices often use different protocols, such as Modbus, Bluetooth, and OPC UA. The mapper abstracts these protocols into a standardized format, allowing applications to interact with devices without needing to know the protocol specifics. Communication happens through a pub/sub model using tools like Mosquitto. In summary, this architecture enables:

● Seamless cloud-edge coordination

● Edge autonomy with offline capabilities

● Low resource consumption

● Simplified device communication through standardization (DMI – we will talk about this later as well)

● An open, flexible ecosystem and this allows devs to build acc. to cloud-native application development while pushing applications to edge devices seamlessly

Project Background

Just a small background about edge computing, So edge computing is bringing the workloads as close to edge as possible, where the data is being created and where the actions are being taken. As networks have evolved, more applications are moving to the edge to improve latency, enable offline autonomy, and enhance data privacy.

Also Edge has its own challenges, Unlike data centers, where you have standardized servers, edge environments often involve diverse hardware—from ARM-based devices to specialized GPUs. Plus, edge networks rely on the public internet, making them very different from controlled data center environments.

As you can see in this slide, how computing services are distributed from the user's device to the central cloud.

Access (ACC) layer - the network segment closest to the user, offering ultra-low latency, typically around 1-5 milliseconds.

Example: The 5G cell tower your phone connects to.

From here, data traffic is consolidated at the **Aggregation (AGG)** layer, a mid-tier segment where latency increases slightly.

Example: A Airtel central office building that gathers traffic from all the cell towers in a specific area

Finally, the **Metro** layer connects these regional networks to the main internet and the central cloud.

Example: A major Internet Exchange Point in a city like Mumbai.

And then how we use edge computing is through - Multi-access Edge Computing (MEC). Instead of sending all data to the cloud, we place compute power at different points along this path, each suited for different tasks:

- **At the closest edge (the ACC layer):** We deploy applications that require near-instantaneous responses. This is perfect for real-time services like **V2X** (Vehicle-to-Everything, which is a communication technology that allows vehicles to interact with its environment for road safety, autonomous driving etc.)

- **At the regional edge (the AGG or Metro layers):** We can run services that benefit from being close to users but require more computing power. Excellent use cases here include **Content Delivery Networks (CDNs)** to stream video without buffering, or compute-intensive tasks like live video transcoding and regional data analytics.
- **In the Central Cloud:** This remains the ideal place for massive-scale, non-latency-sensitive operations like training complex **AI models**, long-term data archival, and large-scale big data processing.

So this is how edge computing reduces the need for long-distance communication by processing data locally and lowering latency.

Issue I worked on

So the Issue I worked on was to write tests for kubeedge. So I wrote unit tests, e2e tests and integration tests for kubeegde. So yes, I wrote a lot of tests, and I got to learn a lot about technical stuff during the mentorship because of that, most important of that being reading a lot of code – this really helped me understand how production level code is written for a complex system like kubeedge. Then I learned about how to write tests – both unit and e2e tests. Before this I had no experience writing tests but once I started working, I understood the importance of test and how to write it, I tried to research on various lib for tests. Learned how to use ginkgo and gomega for BDD. I got to learn how to interact with the Kubernetes api, because for a lot of tests I had to create Kubernetes resoures, so that was really fun.

1. **Learned a little about how KubeEdge works, what it actually does and Edge Computing:**
2. Wrote E2E tests for device plugins, for Deployments, Deployments with probes etc. these were so that we can ensure that the devices can be registered in KubeEdge cluster. And Probes and other Kubernetes resources work fine in the cluster.
3. I also worked on adding senarios to integration tests like I adding device to kubeedge and then adding attributes and twin attributes to it. Some query optimization stuff etc.
4. I also got to interact with the Kubernetes api while creating Deployments and other k8s resources.
5. **Wrote a lot of unit tests**. I also researched and found a better assertion lib for unit tests which is testify/assert. So I saw that there were negligible downsights to that lib and it makes the code much more verbose and easy to write, so I migrate the tests we already had to tesify and since then we have to using testify/assert for unit tests.

Learning

So I got to learn a lot of stuff during my LFX mentorship. I did not know how to write tests for such a complex project before this, so that's the first thing I learned. Other than that I learn how to interact with a k8s cluster using Kubernetes API, I first time used Kubernetes API so that was a lot of fun.

Apart from the technical learnings I talk about, I learned a lot of non technical lessons, that also help me with my current job as an SRE Intern. This are the things that I not only learned while LFX but throughout my journey as an open source contributor.

1. **Just Jump in: You Learn by Doing**

I used to think that you need to know everything about the project and all the tech stack being used in the project just to contribute to it. This is just not true and one can simply jump in to the project – I mean offcourse you should have knowledge of some things – like if you are contributing to some CNCF project then you should know programming basics and then go and little bit of Kubernetes at least.

But apart from that you can just pick some issues and start contributing and learn on the go.

2. **Communication is very important**

I think communication is the most important thing of all, and I see even now when I work as an SRE. Always talk to your maintainers – if you are applying for LFX, ask them what are they exceptiong from a mentee, what resources you should follow etc. and highlight your work at the slack channels.

One more tip here is that almost all the CNCF projects have bi weekly meeting or weekly meetings, where you can talk to the maintainers, set the right expectations and ask questions. Make sure to attend them. And and then there are slack channels, make sure to join them, you can ask your doubts there as well.

3. **It is always like from Abstraction to True Understanding**

Every problem you pick will start like that, you will understand nothing but the problem statement and you slowly slowly understand more stuff and move towards a solutions. So do not be afraid of not understand the ins and outs of a project and do not thing you can't solve an issue without that. You can, just put the work.

4. **Take Ownership**

Maintainers can help you yes, but you will have limited time to ask the project maintinaers so make sure to bring the best questions to them – so before just asking question to a maintainer, first google it – maybe you get the answer there, search in the docs, etc. and then when you ask the maintainer you know that there was no way you would have reached the answer.

Researching before asking also helps you understand better when the maintaines will reply you, because now you understand your question better.

And also do not wait for instructions. Be curious and proactive – take the initiative – if you think something can be improved in the project , then just go ahead, improve it an make PR. Don't just wait for maintainers to give a go before you do something.

5. The Journey is Not a Straight Line

By that I mean that its not like you start your work one the first day of the mentorship and you end it on the last day and everyday In between is same. For example, for me – I had my university exams in middle, so for one month I was managing uni exams and LFX work.

So do not think of the journey as a straight line, some days will be relax, some will be more hectic – but all of them will have a lot of learning for you – so focus on learning :) many of my friends completed their LFX work a month before and relaxed, some went on to contribute more. So the journey is different for everyone and try to make the most out of it.

6. Always say yes to opportunities

I got a chance to speak at Kubecon last year and talk about kubeedge – my maintainers asked me if you wanted to – and I said yes. I didn't knew everything about the project, I was worried how will I answer the questions from the crowd? But still I said yes, and it was an amazing experience. All because I said yes and was ready for the challenge.

So if you get and further opportunites from LFX – just say yes – and later we can figure out how to do it to the best of our abilities. You will get a lot more opportunites if you learn to say yes.

Bonus

Project Journey

KubeEdge started its journey back in 2018 as an open source project, and then it was donated to CNCF in 2019 making it the first cloud native edge project under CNCF sandbox. And then to mention a few milestones, In January 2020, KubeEdge saw large-scale adoption in China's highway systems, A KubeEdge based system for Electronic Toll Collection manages nearly 100,000 edge nodes across multiple provinces of China, definitely with multiple clusters later you will see KubeEdge could support 100,000 Edge Nodes in a single cluster. KubeEdge became an incubation project in 2020.

KubeEdge also launched several sub-projects like Sedna and EdgeMesh, Sedna enhances AI workloads by enabling collaboration between the edge and cloud. EdgeMesh handles communication between edge devices (edge nodes) across different networks.

KubeEdge was used in the first cloud native vehicle for over-the-air updates at the software layer. And then in December 2021, the world's first cloud native satellite based on KubeEdge was launched that verified the cloud-edge integrated solution in space.

So LEO satellites are different from traditional edge devices as they move very rapidly and have very limited connectivity with the ground station. Using KubeEdge, lightweight AI models were deployed directly on the satellite. And these models performed real-time filtering, for example it would discard images that are not useful for processing and only meaningful data will be transmitted back to the ground station. We will talk about this later in our case studies.

And then in June 2022, KubeEdge introduced support for scaling up to 100,000 edge nodes in a single cluster, for meeting the demands of edge computing.

KubeEdge got the SLSA Level 3 certification in 2023 and now it has graduated from CNCF in 2024 october.

DMI

So DMI is by the SIG device IoT and here we focus on decoupling the application development from direct device management. So the idea is “device as a service” so that applications can interact with devices like they interact with Kubernetes services.

The DMI (Device Management Interface) provides a flexible, unified way to manage the IoT devices. This interface enables customized management features, including:

- Device lifecycle actions like creating, updating, and deleting devices.
- Device twin information, reporting real-time events, and managing health checks.
- Over-the-air (OTA) updates, ensuring devices stay current without manual intervention.
- More advanced use cases like device virtualization.

All interactions happen via the Mapper, which acts as a bridge between the device and the application. So when your application runs, it communicates with mapper to fetch the data. The Mapper handles protocols such as Modbus, bluetooth, etc. which we talked about earlier.

Sedna

Sedna is a subproject that enhances AI workloads by enabling collaboration between the edge and cloud. At the Cloud Node, the Global Manager handles:

- Task management and task coordination, ensuring efficient scheduling and execution of AI workloads.

- Model and dataset management, for centralized control over AI models and data.

A Local Controller acts as a bridge between cloud and edge. Each node, both cloud and edge, runs a Worker responsible for tasks such as inference and training.

The Training and inference frameworks that are fueled by this edge cloud collaboration are:

- Joint inference: The cloud and edge collaborate to process AI tasks. The edge handles simple tasks locally, while complex tasks are forwarded to the cloud for processing.
- Incremental learning: Models are continuously updated with new data collected at the edge.
- Federated learning: AI models are trained locally on each edge node, and only the model updates are shared with the cloud, not the raw data. This ensures privacy and security.
- Lifelong learning: Models continuously learn and adapt by processing new, unseen tasks across time.

Currently Sedna supports popular AI frameworks like TensorFlow, PyTorch, and paddle. And also provides extended interfaces for developers to integrate third-party algorithms.

Sedna – Federated Learning

So like I talked about unbalanced data sizes across edge nodes that some edge nodes would have access to larger datasets and some smaller, to tackle this challenge and for ensuring data privacy we have Sedna federated learning service.

In edge AI, it is important to retain the raw data within the edge nodes to protect privacy. The key idea here is that only the model parameters are shared with the cloud, not the raw data.

So how this is done, Raw data stays on each edge node, and model updates are sent to the cloud for aggregation. Developers can train models on the cloud and then push those models to the edge.

And then also thing is Multi-task Detection, since edge nodes handle non-IID sample sets (non-independent and identically distributed), the system divides tasks and collaborates with the cloud to identify and group similar tasks. This helps optimize resource use and improve accuracy.

Joint Inference

So next we will talk about sedna joint inference API, Sedna can be used for Cloud-Edge Joint Inference under resource constrained conditions.

The process works as follows, AI Developers provide the data to train both shallow models at the edge and a deep model in the cloud. Business Developers deploy and call these models using the Sedna Joint-Inference API, which seamlessly integrates cloud and edge inference.

If a request comes in and the confidence level meets a predefined threshold, the shallow model can directly handle the inference and return the result. This is extremely fast because the inference happens locally, close to the user.

However, when the confidence level is not high enough, the system sends the user request to the cloud. In the cloud, a deep model aggregates information from multiple edge nodes, performs the inference with greater accuracy, and returns the result to the user.

So this way developers can ensure a balance between latency and accuracy,

Case Studies - Bonus

Cloud native satellite

So in recent years, the data volume generated by satellites has, for the first time, started exceeding the data transmission volume. Because of which that large amount of data in the satellite needs on-orbit processing, which is a perfect application for edge computing.

In December 2021, KubeEdge was used in making the world's first cloud-native satellite. Basically KubeEdge and the Sedna system was reconstructed acc. To the Low Earth Orbit (LEO) satellite characteristics, for it to have cloud-native edge computing capabilities.

LEO satellites are different from traditional edge devices, they operate in more challenging environments. So they move very rapidly and have very limited connectivity with the ground station, like in a day it will be connected for about 8 to 10 times with the ground station, and this will be for a very short duration, like 6 to 10 minutes per session. Managing data under these conditions is critical. So we need the satellite to become intelligent.

Using KubeEdge, lightweight AI models were deployed directly on the satellite. And these models performed real-time filtering, for example it would discard images which are just covered by clouds which is like 80 to 90% for some satellites. Only meaningful data, like images useful for agriculture or disaster monitoring, etc. are transmitted back to the ground station. This reduced satellite to ground data transmission volume by 90%. And collaborative inference with both large and small models improved the object recognition precision by 50%.

One more interesting use case is that KubeEdge also enabled remote updates and application deployment on the satellite. Once satellites are in orbit, they are beyond physical reach so software updates are essential for longevity and adaptability of the satellite. With KubeEdge, updates and new AI models can be pushed to the satellite.

This would reduce the operational costs, and allow faster data insights which is crucial for cases like floods monitoring and disaster monitoring.

Smart oilfield

Another case study for KubeEdge is offshore oil field operations. Now this is a unique and challenging scenario due to the complex nature of environments like these.

Offshore oil fields have challenges like strict security requirements and device and sensor management. These oil fields rely on a lot of sensors to collect critical data and This data is then processed and analyzed to identify potential risks or anomalies in the system.

Using KubeEdge, these oil companies have been able to simplify and optimize these operations. KubeEdge makes it easier to manage applications deployed on offshore rigs. And AI models deployed through KubeEdge help analyze this sensor data in real-time. This allows for filtering of data in the edge like we saw in the previous case study and this helps in advanced risk analysis.

Also by processing sensitive data at the edge, Kubeedge minimizes the exposure of confidential information to the cloud, which is important for security and compliance.

All this leads to cost reduction and efficiency improvement, Safe production, offline self healing and unified Operations and maintenance management.